The Relay: Architectural Resiliency and Retrieval Report

Candidate Systems Engineering Submission — Repository: https://github.com/ParminderSinghGithub/The-Relay-Vaani

1 What I built and why

We designed and evaluated a resilient question-answering pipeline in Python over three version-pinned arXiv preprints: *Attention Is All You Need* (1706.03762v7), *Moshi* (2410.00037v1), and *Dynamic Chunking (H-Net)* (2507.07955v2). The architecture comprises four stages: (1) **Intent & Scope Classifier (Branching):** Evaluates query scope. Queries outside the three papers branch immediately to an explicit refusal (`refused_correctly`), consuming 0.0 ms retrieval overhead and 0 generation tokens. (2) **Passage Retrieval (Tool Call: `retrieve_passages`):** Searches indexed chunks using TF-IDF lexical scoring with section-aware boosting, returning structured metadata anchors (`[Title, Section, Page]`). (3) **Cited Response Synthesizer:** Generates answers strictly anchored in retrieved passages; if evidence is absent, the model refuses rather than guessing. (4) **Citation & Grounding Verifier (Loop/Feedback):** Verifies citations against source anchors; verification failures trigger a bounded corrective re-prompt (max 1 refinement cycle). **The One Decision That Shaped It Most:** *Prioritizing admission control and verification over optimistic throughput.* When load spikes or quotas deplete, optimistic systems queue indefinitely or hallucinate. Enforcing early intent gating and immediate 0-ms fast-fail on exhausted daily budgets guarantees system predictability and eliminates ungrounded hallucinations.

2 The real service and my stand-in

We probed Groq's hosted API under a free-tier account using `openai/gpt-oss-20b`. Limits were confirmed via live HTTP headers and console telemetry.

Table 1: Limits you found in the real service

Limit	Value you found	How you found it	Confidence (high, medium, low)
Requests per minute	30 RPM	Groq console account limits table	High
Tokens per minute	8,000 TPM	Measured from response header `x-ratelimit-limit-tokens: 8000`	High
Requests per day	1,000 RPD	Measured from response header `x-ratelimit-limit-requests: 1000`	High
Tokens per day	200,000 TPD	Groq console account limits table	High
Requests in flight at once	1–3 concurrent	Empirical observation during paced baseline requests	Medium
What it returns when a limit is hit	HTTP 429 Too Many Requests with JSON error payload and `retry-after`	Official Groq error specification and API telemetry documentation	High
Time until it recovers	Approximately 1.9 s for TPM/RPM; resets at 00:00 UTC for RPD	Live response reset headers (`x-ratelimit-reset-tokens: 1.897s`)	High

Stand-in Behavior and Calibration: Calibrated from live probe measurements (TTFT $\approx 1,868.9$ ms, total latency $\approx 2,278.6$ ms, streaming ≈ 45 tokens/s). Concurrency slowdown models queue contention: $L = L_0 + \alpha \cdot (N_{\text{inflight}} - 1)$ with $\alpha = 180$–220 ms. Rolling 60-second sliding windows enforce RPM/TPM ceilings with authentic HTTP 429 JSON payloads and `retry-after` headers.

Where it differs most from the real thing: The stand-in utilizes deterministic queue math and synthetic text emission. Real hosted infrastructure exhibits multi-tenant GPU queuing, non-linear KV-cache swapping, and reasoning token generation channels.

3 Results

Table 2a: Latency, one row per load scenario (at least: steady, burst, sustained overload, per-minute limit halved, daily budget exhausted)

Scenario	Real or stand-in	Requests sent	Duration (s)	Offered rate (per min)	First token p50 / p95 / p99 (ms)	Total time p50 / p95 / p99 (ms)
Steady (real)	real	5	12	25	1868.9 / 18756.0 / 18756.0	2278.6 / 19103.1 / 19103.1
Steady (stand-in)	stand-in	12	60	12	1728.8 / 1799.0 / 1799.0	2689.0 / 3004.0 / 3004.0
Burst	stand-in	30	3	900	1872.5 / 2217.4 / 2217.4	6003.8 / 6007.0 / 6007.3
Sustained overload	stand-in	45	45	60	1847.4 / 2274.8 / 2274.8	6003.9 / 6010.1 / 6017.4
Per-minute limit halved	stand-in	20	30	40	1998.7 / 1998.7 / 1998.7	6003.5 / 6008.5 / 6008.5
Daily budget exhausted	stand-in	15	5	180	0.0 / 0.0 / 0.0	0.0 / 0.0 / 0.0

Note: TTFT percentiles for Burst/Overload are computed over requests receiving tokens (first_token_ms > 0), while total latency percentiles include all offered requests.

Table 2b: Outcomes, same rows

Scenario	Succeeded %	Rate-limited %	Timed out or given up %	Retries per request	Peak queue length	Tokens per answer	Cost per 1,000 answers
Steady (real)	100.0%	0.0%	0.0%	0.80	1	2236	$0.00
Steady (stand-in)	100.0%	0.0%	0.0%	0.00	1	1803	$0.00
Burst	36.7%	63.3%	0.0%	1.27	15	721	$0.00
Sustained overload	28.9%	71.1%	0.0%	1.42	10	528	$0.00
Per-minute limit halved	30.0%	70.0%	0.0%	1.40	8	670	$0.00
Daily budget exhausted	0.0%	100.0%	0.0%	0.00	2	0	$0.00

Note: Tokens per answer represents total transaction tokens (tokens_in + tokens_out). Free-tier usage incurs $0.00 API cost.

Sample Size Qualification: The live Steady run used a quota-safe sample of 5 real requests; p95/p99 values reflect observed tail behavior within that sample rather than long-run population bounds.

A Number We Could Not Have Guessed Without Running It: The 10x Tail Latency Spike on Real Groq. Requests 1–3 arrived at steady TTFTs of $1,137.3$–$1,868.9$ ms. However, request 5 suffered a massive latency spike to **18,756.0 ms TTFT** and **19,103.1 ms Total Latency** on the live service. Because RAG prompts consumed $\approx 1,800$ tokens each, requests 4 and 5 reached the 8,000 TPM limit at an offered rate of only 25/min (well under the 30 RPM limit), triggering client backoff and multi-tenant batch delay. Across our 16 curated test questions, answer quality achieved **62.5% cited correct, 37.5% correctly refused, and 0.0% wrong**, averaging **999 to 1,466 tokens per answer**.

4 Where the time goes

Examining live Groq request `req_c94d601d` from `results.csv` (Total latency: **19,103.1 ms**, First token: **18,756.0 ms**, Retries: 2, Tokens in: 1,761, Tokens out: 266, Status: `ok`):

- **Delay Before vs After First Token:** Delay occurred overwhelmingly **before the first token** (**18,756.0 ms** before vs **347.1 ms** after, or 98.2% vs 1.8%). Timestamp instrumentation proves dispatch-to-first-chunk elapsed time was $18,756.0$ ms; the remaining 266 tokens arrived over 347.1 ms in rapid burst chunks.
- **Waiting (Client Backoff Sleep):** Client executed exponential backoff with jitter across two retry intervals (backoff $= \min(8.0, 1.0 \cdot 2^{\text{attempt}} + \text{jitter})$), pausing client-side after HTTP 429 rejections while waiting for Groq's 8,000 TPM window to clear.
- **Retrying (Failed Round-trips):** Two intermediate calls were dispatched and rejected upstream with HTTP 429 (TPM exceeded) before attempt 3 succeeded.
- **Working & Instrumentation Boundary:** Local scope classification (< 0.1 ms) and TF-IDF retrieval (0.8 ms) took < 15 ms combined. The final attempt was admitted by Groq, emitted reasoning tokens, and finished in 347.1 ms. Because intermediate attempt round-trips were aggregated into the request summary rather than isolated in sub-millisecond trace spans, we report the pre-first-token delay ($18,756.0$ ms) as an aggregate rate-limit congestion and backoff penalty, labeling unmeasured network transit as unexplained overhead rather than inferring unverified component splits.

5 When a limit is hit

- **At the Per-Minute Limit (RPM/TPM):** The pipeline throttles requests using exponential backoff with full random jitter (backoff $= \min(4.0, 1.0 \cdot 2^{\text{attempt}} + \text{jitter})$), honoring upstream `retry-after` headers. In-flight interactive queries are delayed. If congestion persists past 2 retries, requests are dropped cleanly with status `rate_limited`. Across the 80 rate-limited outcomes in `results.csv`, 65 represent load-shed drops on valid queries (graded `refused_wrongly` due to capacity exhaustion).
- **At the Per-Day Limit (RPD/TPD):** All queueing and retrying is bypassed entirely. The pipeline fast-fails incoming requests at Step 1 with zero latency (0.0 ms), immediately notifying the user. These 15 daily budget requests are graded `refused_correctly`.
- **Who gets a weaker answer:** When available TPM drops below 2,500 tokens, retrieval clamps chunk retrieval from 4 pieces down to 1 piece ($k = 1$), halving prompt token burn to preserve service availability.

Retrieval: quality versus cost

Table 3: Retrieval settings (at least three)

Setting (piece size, pieces fetched, re-rank yes/no)	Questions asked	Correct and cited %	Wrong %	Correctly refused %	Tokens per answer	First token p50 / p95 (ms)	Total time p50 / p95 (ms)
Setting A (200t, 6 pieces, no re-rank)	16	62.5%	0.0%	37.5%	1134	1689.2 / 1793.6	2555.0 / 2847.1
Setting B (500t, 4 pieces, re-rank yes)	16	62.5%	0.0%	37.5%	1466	1638.5 / 1802.0	2555.3 / 2971.9
Setting C (1000t, 2 pieces, no re-rank)	16	62.5%	0.0%	37.5%	999	1641.7 / 1801.5	2632.7 / 2895.9

Chosen Setting & Justification: We select **Setting C (1000 tokens, 2 pieces, no re-ranking)**. Setting C achieves identical 100% factual accuracy (62.5% correct, 37.5% correctly refused) while consuming **31.9% fewer tokens** than Setting B (999 vs 1,466 tokens/answer) and avoiding equation fragmentation. When approaching the per-minute limit, retrieval dynamically clamps to $k = 1$, reducing prompt overhead to ≈ 500 tokens.

How this library will mislead a naive retriever

1. **Shared Terminology Trap (Multi-Head Self-Attention):** *Query:* 'Explain the multi-head self-attention mechanism used in Moshi's Helium text backbone." *Failure:* Lexical matching pulls Section 3.2 of Vaswani et al. (1706.03762) instead of Moshi's Helium model (2410.00037, Section 3.2), attributing Transformer 64-dimensional parameters to Moshi.
2. **Misplaced Technique Trap (Dynamic Chunking Threshold):** *Query:* 'What dynamic chunking threshold is used in Attention Is All You Need?" *Failure:* Naive search independently matches keywords from 1706.03762 and hallucinates a threshold instead of recognizing dynamic chunking was introduced in H-Net (2507.07955) and is absent from 1706.03762.
3. **Implicit Elimination Trap (BPE Vocabulary Size):** *Query:* 'What is the BPE vocabulary size of the H-Net dynamic chunking model?" *Failure:* Lexical matching extracts baseline comparison passages in 2507.07955, reporting baseline vocabulary sizes rather than understanding H-Net eliminates tokenizers and operates on raw bytes.

6 Retries

Our policy applies exponential backoff with full random jitter capped at 2 retries and an 8.0-second maximum ceiling: backoff $= \min(8.0, 1.0 \cdot 2^{\text{attempt}} + \text{random}(0.01, 0.05))$, adapting dynamically to upstream `retry-after` headers.

One case where retrying made things worse: In the Burst scenario (30 concurrent requests), 19 requests tripped the 8,000 TPM limit on attempt 0. Because clients backed off for only 1–2 seconds and retried before the 60-second sliding token window had cleared, retries slammed back into the same rate limit, amplifying peak queue length to 15, consuming thread pool workers, and adding 6 seconds of wasted latency per request before failing.

7 What the person sees

At 3:00 PM when daily quota is exhausted, waiting users receive an immediate, transparent notice: *'The daily query budget for this research library assistant has been exhausted for today (1,000 requests/day limit reached). Quotas reset at 00:00 UTC. For urgent inquiries, please consult arXiv preprints 1706.03762, 2410.00037, and 2507.07955 directly."*

8 What I'm unsure about, what I left out, what I learned

- **Least Certain Assumption:** Assuming TF-IDF lexical search with section boosting is sufficient across dense ML papers. Complex mathematical derivations can span chunk boundaries, causing potential citation misses without dense semantic vector search.
- **Two Deliberately Omitted Features:** (1) Dense vector embedding databases (Chroma/Pinecone), keeping the system lightweight and zero-dependency. (2) Unbounded client retry queues, preventing artificial tail latency under sustained overload.
- **One Thing Learned (In My Own Words):** *End-to-End Hierarchical Dynamic Chunking (H-Net).* Standard language models rely on heuristic BPE subword vocabularies. H-Net replaces static tokenizers by learning dynamic boundary segmentation directly within the neural architecture using a chunking layer and ratio loss. This processes raw bytes directly, matching token-based compute while dramatically improving efficiency on non-subword modalities.

Your feedback on Vaani

Reviewing `https://www.vaani.dev/`:

- **What Stood Out:** The clear ambition to solve real-time full-duplex conversational voice AI, featuring turn-taking, natural pauses, and sub-second voice turnaround (≈ 450 ms p90 target).
- **What Felt Slow** / **Observed Friction:** During firsthand interaction with the platform, overall responsiveness felt slow. While voice AI promises instant turnaround, interactive response times felt noticeably sluggish. In voice applications, even a 500–800 ms hesitation feels like awkward dead air, disrupting natural conversational cadence.
- **One Thing I Would Change:** Provide immediate front-end feedback or optimistic audio indicators (such as conversational filler utterances like 'mm-hmm" or an instant visual speech visualizer) as soon as voice input stops, paired with edge-routed WebSocket streaming to prevent initial dispatch latency from degrading perceived conversational fluidity.

How I worked with the assistant

Conversation Milestones: (1) Corpus setup with 3 arXiv preprints and privacy guards on `.env`. (2) Real Groq baseline calibration ($\approx 1,868.9$ ms, tail $\approx 18,756.0$ ms, limits: 8,000 TPM / 1,000 RPD). (3) Implementation of calibrated stand-in and 4-step pipeline. (4) 16 evaluation questions and 5 load scenarios in `results.csv`, generating Tables 1, 2a, 2b, 3.

Three Prompts That Mattered Most: (1) *'please check groq's current official docs and my account's model limits... don't waste the free quota trying to hit limits. one thing: don't touch or read my .env file... plan the required pipeline, groq integration, stand-in service, five load scenarios, 15+ evaluation questions, three retrieval configurations, metrics and exact results.csv format."* (Protected credentials, prevented quota burn, anchored baseline calibration). (2) *'the plan looks good proceed with implementation... fix the report outline so all 11 headings from the assignment are in the exact required order keep the implementation simple reproducible and within assignment constraints... clearly separate documented facts measured results and assumptions..."* (Guaranteed structural alignment to assignment rubric and strict heading order). (3) *'Check results.csv against the actual benchmark logs and code. Investigate the identical real-service TTFT/total-latency percentiles, verify all five real probe requests are represented correctly, and confirm every row represents an actual request rather than a manually constructed or aggregated record..."* (Uncovered unpaced saturation and reasoning token channel, enforcing empirical trace integrity).

One Error I Caught: The assistant initially conflated successful p50 TTFT ($1,847.4$ ms) with failed p50 total latency ($6,003.9$ ms) under a non-existent request ID, asserting that the request waited $4,980.2$ ms in backoff sleep yet received its first token at $1,847.4$ ms. I caught this contradiction, forced it to trace `results.csv`, and reconstructed a consistent timeline on actual live request `req_c94d601d`.

One Decision Where I Overruled It: The assistant wanted to write the final report based on passing unit tests alone. I overruled it, mandating an exhaustive forensic audit across all 11 headings, CSV rows, citations, and latency accounting before authoring the report.

What in This Submission Is Distinctly Mine: The audit and verification philosophy; admission control and 0-ms rejection on daily quota exhaustion; the 16-question evaluation suite targeting genuine paper nuances; firsthand evaluation that Vaani felt slow; and strict enforcement of empirical traceability over synthetic claims.